

Document Individuel — Alexis Ballenghien

Projet: **CineMatch** (React Native / Expo, M1 DEV SUP de VINCI)

Partie A — Ma contribution au projet

J'ai realise le projet en solo, donc j'ai developpe l'ensemble des ecrans et de la logique metier:

- **Authentification (Supabase):** inscription, connexion, deconnexion, gestion de session.
- **Explore / Swipe:** affichage des films, swipe gauche/droite, persistence des choix.
- **Matches:** liste des films likes, tri et rafraichissement.
- **History:** historique des swipes (likes/rejets) avec filtres.
- **Search:** recherche TMDb avec debounce (500 ms), pagination et etats vides.
- **Detail film:** synopsis, genres, runtime, casting (top 5), lien bande-annonce.
- **Bonus:** filtre par genre + animations Reanimated sur la carte (rotation/feedback).

Difficultes techniques rencontrees et resolutions

1. Flux signup Supabase

Au depart, la verification email bloquait l'accès immédiat à l'app.

Solution: ajustement de la configuration Supabase pour garder un parcours de connexion coherent avec l'exercice, puis simplification du flux cote app.

2. RLS et creation du profil utilisateur

Certains utilisateurs n'avaient pas de profil apres l'inscription, ce qui cassait les ecritures en base.

Solution: creation de profil systematique apres signup et alignement des policies RLS avec les operations attendues.

3. Recherche trop couteuse sans debounce

Un appel API partait a chaque caractere.

Solution: debounce a 500 ms + gestion claire des etats (chargement, vide, erreur, resultat).

4. Restauration de progression du swipe

L'utilisateur revenait au debut apres rechargement.

Solution: reconstruction de l'index depuis l'historique persiste et rechargement progressif des pages.

Ce que j'aurais fait differemment avec plus de temps

- Ajouter des tests automatisés (unitaires + parcours E2E).
- Renforcer la couche de monitoring/observabilite des erreurs runtime.
- Ajouter des notifications push et du temps reel pour enrichir l'experience.
- Travailler une phase onboarding plus guidee.

Partie B — Mon utilisation de l'IA

Outils d'IA utilises

- **GitHub Copilot** (principal): generation de boilerplate, suggestions TS/React Native, refactor.
- **ChatGPT** (ponctuel): comparaison d'approches et reformulation de prompts techniques.

Exemples concrets de prompts et resultats

Exemple 1 — Hook de stack de films

Prompt utilise:

```
Create a custom React hook to manage a movie stack with:  
- pagination  
- genre filtering  
- restoring current index from swipe history  
Return state + callbacks for loading next pages.
```

Ce que l'IA a produit: une base de hook exploitable (state, refs, callbacks).

Ce que j'ai corrige: la logique de reprise de progression et certains effets/dependances pour eviter les incoherences.

Exemple 2 — Animation de swipe

Prompt utilise:

```
Generate Reanimated code for a swipe card:  
- rotate while dragging  
- spring back when below threshold  
- green/red feedback opacity based on direction
```

Ce que l'IA a produit: structure Reanimated correcte (shared values + animated style).

Ce que j'ai corrige: reglages des seuils/valeurs et adaptation aux types utilises dans mon ecran.

Exemple 3 — Debounce recherche

Prompt utilise:

```
Implement debounced search in React Native with:  
- 500ms delay  
- loading, empty, error states  
- pagination reset when query changes
```

Ce que l'IA a produit: un squelette utile pour la logique de debounce.

Ce que j'ai corrige: orchestration des etats UI et reinitialisation propre des resultats/pagination.

Ce que l'IA a bien fait / mal fait

Bien fait

- Accelération nette sur le boilerplate et la structure initiale.
- Bonne aide sur les patterns repetitifs (hooks, etats, composants).
- Gain de temps sur les ajustements TypeScript simples.

Moins bien fait

- Propositions parfois trop generiques pour la logique metier reelle.
- Quelques suggestions techniquement valides mais pas adaptees au contexte du projet.
- Besoin de verification systematique (noms, typage, perf, coherence UX).

Reflexion personnelle

L'IA m'a fait passer d'une logique "coder puis corriger" a une logique "specifier, evaluer, adapter". Concretement, j'ecris des prompts plus precis, je garde le controle sur les decisions, et j'utilise l'IA comme accélérateur de production — pas comme remplacement de la conception. Sur ce projet, cela m'a surtout aide a aller plus vite sur les fondations, pour consacrer plus de temps a la qualite fonctionnelle et a la coherence globale de l'app.